

Comparative Study of Machine Learning Approaches for Physical Simulation Data: Tree-Based Models Outperform Neural Networks

Tommer Weizmann¹

Abstract

This study investigates the effectiveness of three machine learning models—Random Forest Regressor, XGBoost, and Multilayer Perceptron (MLP)—in predicting physical simulation data governed by partial differential equations. The primary aim is to identify an optimal model that minimizes prediction error while maximizing computational efficiency. Unlike physics-informed neural networks (PINNs), which incorporate physical laws during training, this study relies solely on data-driven learning using simulation data generated from the heat equation. XGBoost and Random Forest achieved mean absolute percentage errors of 1.91% and 2.43%, respectively, with inference times under one second. These results are comparable to published PINN models for similar problems, suggesting that tree-based models can offer efficient alternatives when computational resources are limited. Future work may explore hybrid physics-informed tree-based models that combine the efficiency of data-driven learning with the robustness of physical knowledge.

Keywords: Machine Learning, Tree Based models, Neural Networks, XGBoost, Random Forest, Physics-Informed Neural Networks (PINNs), Surrogate modelling, Partial Differential Equations, Computational Efficiency, Engineering Applications

1. Introduction

In engineering and applied sciences, generating computational results based on physical models often requires significant time, ranging from hours to days. This computational expense limits rapid design iteration and real-time decision-making. Consequently, there is a growing need for surrogate modeling techniques that leverage machine learning to provide fast, accurate, and trustworthy predictions as alternatives to full-scale simulations.

The primary objective of this study is to identify an optimal machine learning algorithm capable of delivering accurate predictions with high computational efficiency, without resorting to advanced and computationally demanding approaches such as Physics Informed Neural Networks (PINNs). Instead, this work focuses on widely accessible algorithms including Random Forest Regressor, XGBoost, and Multilayer Perceptron (MLP).

All models were trained on data generated by a 3D heat conduction solver based on the

solution developed by [4] using Green's functions. The study assumes that these findings can generalize to other physics and engineering problems of similar complexity, where surrogate modeling can substantially reduce computation time while maintaining accurate predictions.

2. Mathematical Model

The heat transfer in the system is modeled using the analytical solution of the heat equation, as presented by [4]. The temperature distribution $T(x, y, z, t)$ at position (x, y, z) and time t is given by:

$$T(x, y, z, t) = \int_{\mathbb{R}^3} G(\mathbf{x} - \mathbf{y}, t) T_0(\mathbf{y}) d\mathbf{y} + \int_0^t \int_{\mathbb{R}^3} G(\mathbf{x} - \mathbf{y}, t - t') f(\mathbf{y}, t') d\mathbf{y} dt', \quad (1)$$

where:

- $T(x, y, z, t)$ is the temperature field,
- $G(\mathbf{x} - \mathbf{y}, t)$ is the three-dimensional Green's function of the heat equation, defined as:

$$G(\mathbf{x} - \mathbf{y}, t) = \frac{1}{(4\pi\alpha t)^{3/2}} \exp\left(-\frac{|\mathbf{x} - \mathbf{y}|^2}{4\alpha t}\right), \quad (2)$$

- α : thermal diffusivity of the material,
- $T_0(\mathbf{y})$: initial temperature distribution,
- $f(\mathbf{y}, t')$: heat source term representing a pulsed Gaussian laser.

In practice, the simulation domain is finite, so numerical integration is performed over a bounded volume.

The source term is modeled as:

$$f(x, y, z, t') = \frac{\mu_a}{\rho c} I_0 \exp\left(-\frac{(x - x_0)^2 + (y - y_0)^2}{w^2}\right) \times \exp\left(-\frac{z}{\delta}\right) \exp\left(-\frac{(t' - t_0)^2}{2\sigma^2}\right), \quad (3)$$

with parameters:

- μ_a : absorption coefficient (m^{-1}),
- ρ : material density (kg m^{-3}),
- c : specific heat capacity ($\text{J kg}^{-1} \text{K}^{-1}$),
- I_0 : peak laser intensity (W m^{-2}),

- (x_0, y_0) : center of the laser beam in the xy -plane (m),
- w : beam radius (m),
- δ : laser penetration depth (m),
- t_0 : pulse center time (s),
- σ : pulse duration parameter (s).

2.1. Boundary Conditions

Two types of boundary conditions are applied:

1. **Neumann (insulated) boundary condition**, enforcing no heat flux across the boundary:

$$\frac{\partial T}{\partial \mathbf{n}} = 0,$$

where $\mathbf{n}$ is the outward normal vector to the surface.

2. **Convective boundary condition** at the surface $z = 0$, modeling heat loss to the environment using Newton's law of cooling.

2.2. Computational Solution

The heat equation is solved numerically using an explicit finite difference method. The temperature distribution is given by:

$$T^{n+1} \approx T^n + \Delta t \left(\alpha \nabla^2 T^n + f(x, y, z, t) \right). \quad (4)$$

To include convective cooling at the top surface ($z = 0$), the update rule is modified as:

$$T^{n+1} \approx T^n + \Delta t \left(\alpha \nabla^2 T^n + f(x, y, z, t) - \frac{h}{\rho c \Delta z} (T^n - T_\infty) \cdot \chi_{z=0} \right) \quad (5)$$

where:

- h : convective heat transfer coefficient ($\text{W m}^{-2} \text{K}^{-1}$),
- T_∞ : ambient temperature ($^\circ\text{C}$),
- Δz : grid spacing in the z -direction (m),
- $\chi_{z=0}$: indicator function equal to 1 on the $z = 0$ surface and 0 elsewhere.

This model accounts for heat distribution including convective heat loss at the material's top surface. The indicator function $\chi_{z=0}$ ensures that the convective cooling term applies only at the $z = 0$ boundary. Other boundary conditions can be incorporated similarly to model heat flux on different surfaces, which may influence the overall heat loss observed in the computational results.

3. Machine Learning Models

Three machine learning models were selected for this study: Random Forest, XG-Boost, and Multi-Layer Perceptron (MLP). These models were chosen due to their ease of implementation and their ability to capture nonlinear relationships between input features and target variables. Nonlinear interactions are common in physics and engineering applications, making these models suitable for accurate prediction tasks in such domains.

All models were trained using default hyperparameters provided by their respective libraries, reflecting a realistic scenario where ease of implementation and minimal setup effort are prioritized. The exception was the Multilayer Perceptron (MLP), for which the number of training iterations was increased to 15,000 to explore its learning behavior and ensure convergence. No systematic hyperparameter tuning was performed, as the primary focus of this study is to evaluate baseline performance and efficiency of the machine learning models.

3.1. Random Forest Regressor

Random Forest is an ensemble learning algorithm that improves prediction accuracy and robustness by combining multiple decision trees. Each tree is trained on a randomly sampled subset of the data (with replacement), and at each split, only a random subset of features is considered. For regression tasks, the final prediction is the average of the outputs from all individual trees. This method reduces overfitting, enhances generalization, and effectively models complex, nonlinear relationships in high-dimensional datasets [2].

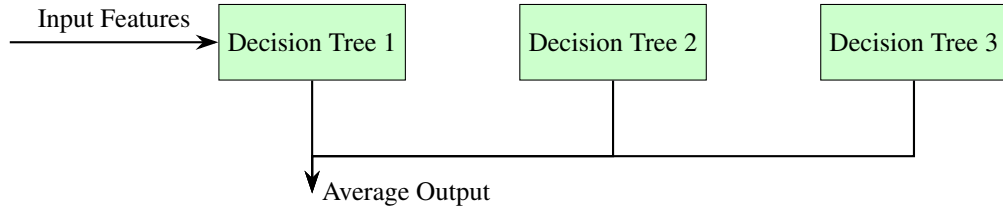


Figure 1: Illustration of how the Random Forest Regressor works: input features pass through multiple decision trees, each producing a prediction; the final output is the average of these predictions.

3.2. Multilayer Perceptron

A Multilayer Perceptron (MLP) is a type of feedforward artificial neural network composed of multiple layers of interconnected neurons. Each neuron computes a weighted sum of its inputs and applies a non-linear activation function, enabling the network to capture complex, nonlinear relationships that often appear in physical systems. Unlike traditional physics-based models, which rely on explicit equations and assumptions, MLPs learn patterns directly from data, making them especially useful when the underlying physical relationships are difficult to express mathematically or computationally expensive to solve. Through training through backpropagation, MLPs can approximate intricate mappings between input features and output variables in regression tasks [1].

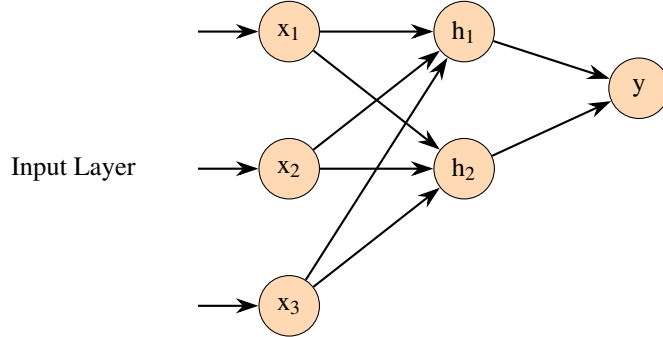


Figure 2: Illustration of a feedforward neural network. Input features are passed into the input layer, processed by the hidden layer, and produce a final prediction at the output layer

3.3. XGBoost Regressor

XGBoost (Extreme Gradient Boosting) is an efficient and scalable implementation of gradient boosting machines that constructs an ensemble of decision trees in a sequential manner. Each subsequent tree aims to correct the errors of its predecessor by optimizing a differentiable loss function using gradient descent techniques. This method incorporates regularization to prevent overfitting and supports parallel processing, making it particularly effective for modeling complex, nonlinear relationships in high-dimensional datasets. XGBoost has been widely adopted in various domains, including engineering and physical sciences, for tasks such as regression and classification, due to its superior predictive performance and computational efficiency [3].

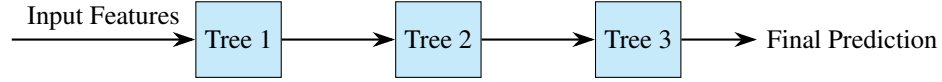


Figure 3: Illustration of XGboost. XGBoost creates trees one after another, each fixing mistakes from the last to make better predictions..

4. Experimental Setup

To train the three machine learning models, XGBoost, Random Forest, and Multi-layer Perceptron (MLP), a data set was generated by numerically solving Equation (5), representing a heat transfer scenario governed by a partial differential equation. Key physical parameters were systematically varied across realistic ranges:

- **Power:** 0.1 W to 1 W
- **Beam Radius:** 1×10^{-7} m to 1×10^{-3} m
- **Domain Length (z):** 0.001 m to 1.5 m
- **Wavelength:** 400×10^{-9} m to 800×10^{-9} m

- **Object Radius:** 0.01 m to 1 m
- **Material Radius:** 1 cm to 3 cm
- **Materials:** Gold, Aluminum, Chromium, Iron, Platinum, Brass, Lead, Silver, Magnesium, Titanium, Copper, Zinc, Tungsten, Steel, Nickel, Stainless Steel, Bronze
- **Geometry Type:** Cartesian and Cylindrical

The material types were encoded programmatically by linking each material name to a corresponding set of physical properties using a Python dictionary. This approach enabled the inclusion of material-specific numerical features (density, thermal conductivity, specific heat capacity).

In addition to these physical parameters, several derived features were computed and included as model input. These included:

- Initial intensity (W m^{-2})
- Total energy inserted (J)
- Penetration depth (m)
- Optical absorption coefficient (m^{-1})
- Volume and surface area (m^3 , m^2)
- Surface-to-volume ratio (m^{-1})
- Peak laser source term (K s^{-1})
- Cooling rate (K s^{-1})
- Time to reach maximum temperature (s)

In addition, descriptive statistics of the resulting temperature fields, such as mean, median and maximum temperature in the 90th percentile, were included to enrich the feature set.

Non-physical outliers were removed during preprocessing. Specifically, samples where the maximum temperature exceeded 10000°C were discarded, resulting in a cleaned data set of approximately 2,400 valid samples. The data set was randomly divided into 80% for training and 20% for testing. All three models were trained using the same data set and evaluation pipeline to ensure a fair comparison. Feature importance was assessed to interpret each model's behavior, learning curves were used to evaluate convergence and generalization, and residual analysis identified patterns of underperformance.

The performance of the model was evaluated using the following metrics:

- Coefficient of determination (R^2)
- Mean absolute error (MAE)

- Training and inference time (in seconds)

To test generalization, each model was evaluated on a new dataset generated using the same equation but restricted to a different subset of materials. The following additional metrics were computed:

- Mean Absolute Error (MAE)
- Mean absolute percentage error (MAPE)
- Coefficient of determination R^2
- Time taken to make predictions

This allowed a review of the ability of each model to extrapolate to unseen, but physically consistent, data.

5. Results

5.1. Overview and Initial Tests

Initially, the three models were compared to evaluate their ability to predict the maximum temperature from simulation inputs, and the overall quality of their predictions.

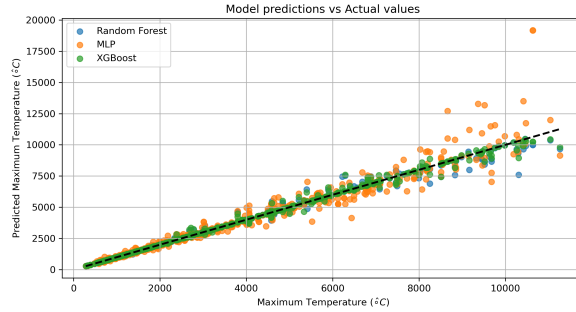


Figure 4: Comparison of predicted vs. actual maximum temperatures for Random Forest, XGBoost, and MLP models.

Table 1: Performance comparison of the three trained models.

Model	R^2	MAE	Time (s)
Random Forest	0.990	96.6	2.79
MLP Regressor	0.920	335	3.59
XGBoost	0.990	79.0	0.324

As shown in Figure 4, all models fit the test data well overall. This is supported by Table 1, which shows strong R^2 values near 1 for both Random Forest and XGBoost.

Additionally, Table 1 demonstrates that XGBoost requires significantly less time for training and testing compared to Random Forest and MLP. Considering that generating this amount of data takes hours, these results indicate highly accurate predictions achieved in minimal time.

For practical application of this work, it is important to identify the error in predictions to account for variations from expectations. In ongoing work, it has been observed that the absolute error tends to increase exponentially with maximum temperature, such that

$$\text{Absolute Error} \propto T_{\max}^r$$

where r is a dimensionless exponent representing the rate of increase of the error. By plotting a log-log plot, the gradient of the fitted line provides the value of r , allowing us to quantify the error on predictions where

$$\text{Absolute Error} = |T_{\text{prediction}} - T_{\text{actual}}|.$$

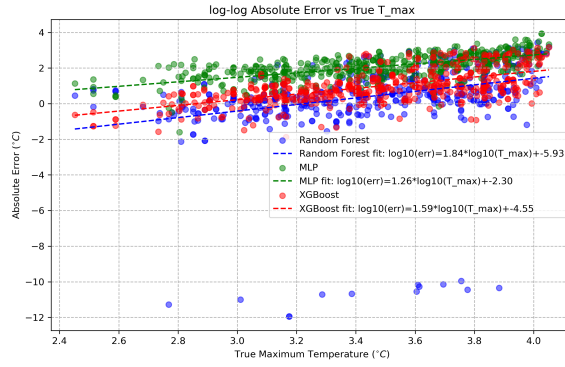


Figure 5: Log-log plot of the absolute error against the maximum temperature.

As shown in Figure 5, the maximum error follows the expected relationship with a linear fit on the log-log scale for each model. The values of r for each model are: 1.84 for Random Forest, 1.26 for MLP, and 1.59 for XGBoost.

5.2. Random Forest Results

The Random Forest model demonstrates strong interpretability. The dominant feature is the source term, which aligns with physical expectations, as it transfers energy into the cube and slab surfaces, thereby increasing temperature. This supports the theoretical relationship $T \propto f(x, y, z, t')$, where $f(x, y, z, t')$ represents the source term.

Additionally, the model recognizes the significance of the time at which the maximum temperature occurs, suggesting that temporal factors are closely linked to temperature peaks. Other influential features are largely geometric—such as length and surface-to-volume ratio—which is consistent with expected behavior for moderate temperature increases.

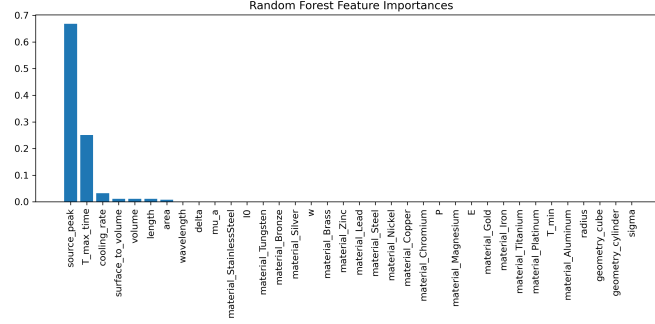


Figure 6: Feature importance derived from the Random Forest model.

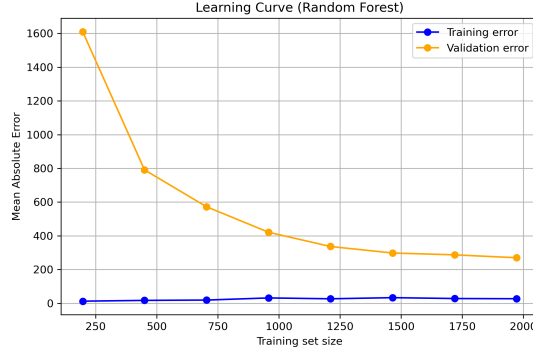


Figure 7: Learning curve of the Random Forest model, showing MAE as a function of training set size.

To quantify the learning rate, the Mean Absolute Error (MAE) is modeled as a function of training set size N :

$$MAE(N) = \exp(-\alpha N) + C$$

From the learning curve in Figure 7, when $MAE \approx 1600$ °C and $N = 250$, with $C = 200$, we solve:

$$1600 = \exp(-\alpha \cdot 250) + 200 \quad (6)$$

$$\Rightarrow \alpha \approx \frac{-\ln(1400)}{250} \approx 0.0290 \quad (7)$$

Thus, the estimated learning rate of the Random Forest model is approximately 0.0290. The residuals plot in Figure 8 shows that while most data points remain close to zero, a consistent pattern of minor deviations exists. This suggests that the model avoids both overfitting and underfitting, and is likely to generalize well to unseen data.

5.3. XGBoost Results

The XGBoost model places emphasis on similar features as the Random Forest model. Notably, it assigns the highest importance to the time at which the maxi-

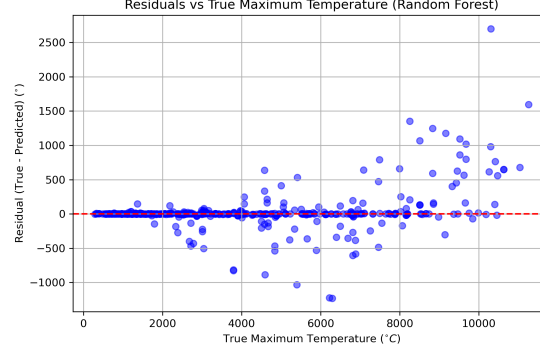


Figure 8: Residual distribution across the temperature domain for the Random Forest model.

imum temperature is reached, followed closely by the source term. This alignment with the Random Forest model suggests that both algorithms have identified consistent and physically meaningful patterns in the dataset.

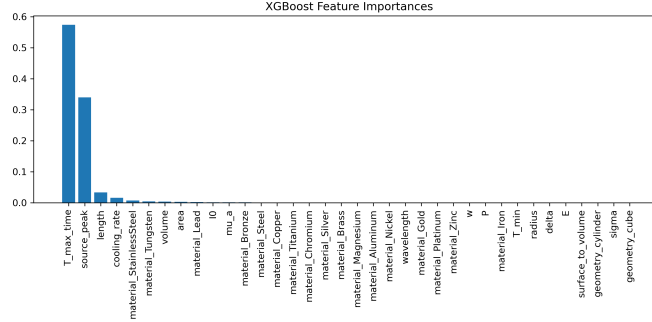


Figure 9: Feature importance values from the XGBoost model.

The learning curve of the XGBoost model is similar in shape to that of the Random Forest model, although it appears to converge more quickly. To quantify the learning rate, we apply the same method used previously. From the curve shown in Figure 10, when the error is approximately 1750 and the training set size is 250, we estimate:

$$1750 = \exp(-250\alpha) + 250 \quad (8)$$

$$\Rightarrow \alpha = -\frac{\ln(1500)}{250} \approx 0.293 \quad (9)$$

This suggests that the XGBoost model's learning rate is approximately the same as that of the Random Forest algorithm. As shown in Figure 11, most residuals remain close to zero. For those that deviate, the errors appear to be evenly distributed across the range, indicating that the model generalizes well—similarly to the Random Forest model.

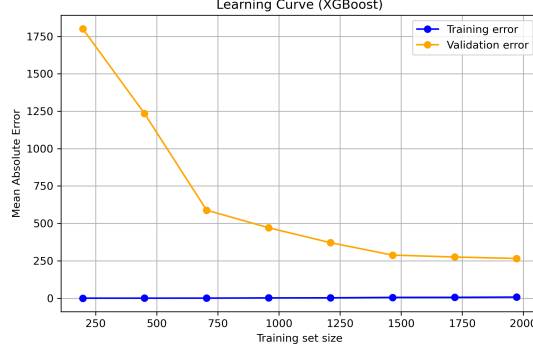


Figure 10: Learning curve of the XGBoost model.

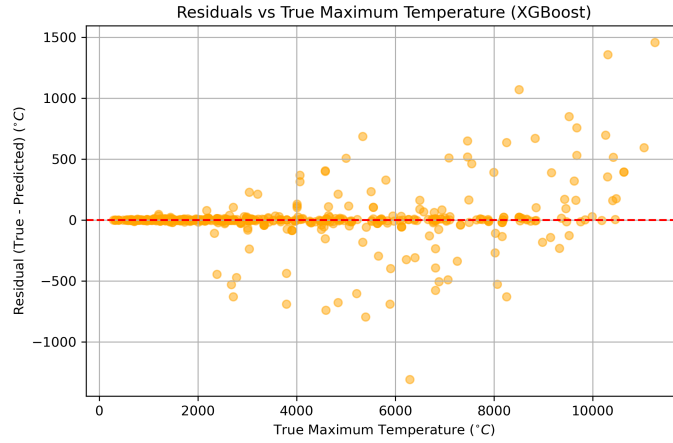


Figure 11: Residuals plot for the XGBoost model.

5.4. Multi-Layer Perceptron

The interpretation provided by the MLP model offers an interesting and somewhat unexpected perspective on feature importance. It places the greatest emphasis on the surface-to-volume ratio, which is physically plausible. Since the laser interacts primarily with the surface, a higher surface-to-volume ratio may result in more intense surface heating and elevated maximum temperatures. Conversely, a lower ratio allows greater heat dissipation into the volume, reducing the temperature rise.

The second most important feature is the beam radius w , which also aligns well with physical intuition. A smaller beam radius focuses energy into a narrower area, leading to higher localized temperatures. Notably, the dataset required preprocessing to remove unrealistic temperature values caused by very small beam radii prior to training.

The model also assigns considerable importance to material type. For example, ma-

materials such as gold, aluminum, chromium, iron, and platinum are considered more influential than steel, stainless steel, or tungsten. This distinction reflects real-world differences in thermal conductivity and absorptivity under laser exposure. However, a point of concern is that the MLP assigns no significant importance to the source term—a key physical parameter directly related to energy input. This raises questions about the internal logic the model has developed and suggests potential over-fitting or feature interaction effects.

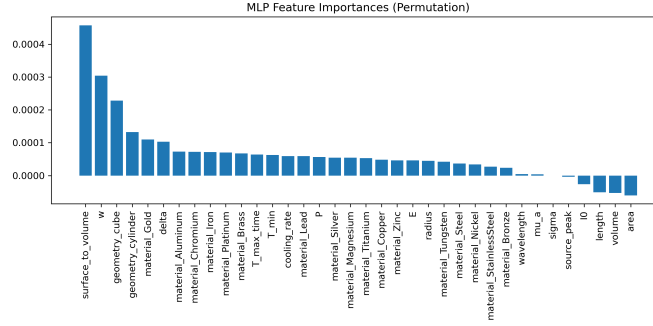


Figure 12: Feature importance values from the MLP model. Since MLP does not provide native feature importances, permutation-based analysis was used.

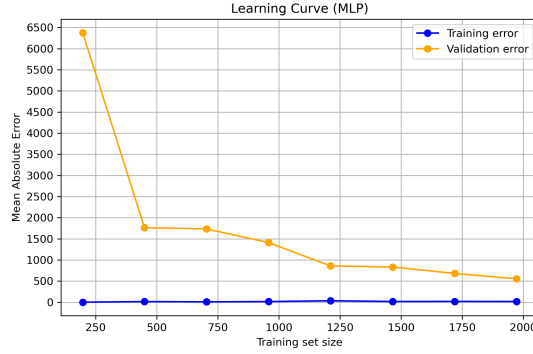


Figure 13: Learning curve of the MLP model.

As shown in Figure 13, the MLP learning curve exhibits an initially sharp decrease in error, followed by a long period of slow improvement. This behavior is characteristic of neural networks, which may require large amounts of data and iterations to converge effectively. Although the curve differs significantly from that of the Random Forest and XGBoost models, a negative exponential trend can still be used to estimate the learning rate. At a training set size of 250, the mean absolute error is approximately 6500:

$$6500 = \exp(-\alpha \cdot 250) + 500 \quad \Rightarrow \quad \alpha \approx \frac{-\ln(6000)}{250} \approx 0.00435$$

This gives an estimated learning rate of $\alpha = 0.00435$. While this value appears lower than the other models, the MLP required approximately 15,000 training iterations to reach it. This suggests slower convergence in practice, despite the comparable final error.

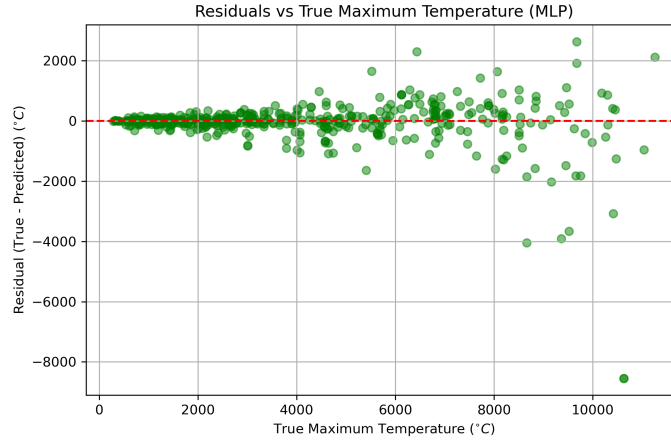


Figure 14: Residuals for the MLP model, showing deviation at temperature extremes.

As shown in Figure 14, the residuals remain small for lower temperature predictions, but grow significantly at higher temperatures. This pattern indicates potential overfitting to mid-range values and weaker generalization for extreme cases. As such, the MLP model may be less reliable in extrapolative scenarios compared to the tree-based models.

5.5. Model Testing on New Data

The trained models were subsequently evaluated using a new dataset to assess their generalization performance. Table 2 summarizes the performance metrics and prediction times for each model. Among the three, the XGBoost model achieved the best results, exhibiting the highest R^2 value and lowest error metrics. Conversely, the MLP model performed poorly, with error values indicating unreliable predictions on the new data.

Model	MAE (°C)	MAPE (%)	R^2	Time (s)
Random Forest	121	2.43	0.992	0.126
MLP	45600	980	-2880	0.0471
XGBoost	98.1	1.91	0.994	0.132

6. Error Comparison Against Physics-Informed Neural Networks (PINNs)

From Table 2, the XGBoost model achieved a mean absolute percentage error (MAPE) of 1.91% in 0.132 seconds. This section contextualizes these results by comparing them to published studies employing physics-informed neural networks (PINNs) for heat transfer simulations.

[5] applied PINNs to predict the 3D temperature field during laser metal deposition, achieving a maximum relative error of approximately 2% compared to finite element method results. [7] compared PINN predictions to OpenFOAM simulations in a 3D printing heat transfer setting, reporting relative temperature errors below 5% across most regions, with higher discrepancies near boundaries. [6] introduced a transfer learning-based PINN capable of reconstructing a full 3D temperature field from a single 2D thermal snapshot in a directed energy deposition (DED) scenario, achieving an average temperature error below 1.3%.

Overall, PINN-based models typically report errors ranging between 1% and 5% when compared to simulation data. While PINNs demonstrate strong potential for physics-informed modeling, these results suggest that simpler and more efficient machine learning models, such as XGBoost, can achieve comparable accuracy. Furthermore, XGBoost offers significant advantages in computational speed and ease of implementation, making it a practical choice for engineering applications that demand rapid inference and efficient resource utilization.

7. Discussion

The results demonstrate promising performance: the tree-based models XGBoost and Random Forest successfully learned to approximate the underlying partial differential equation by predicting the maximum temperature reached during each heat simulation. They achieved mean absolute percentage errors of 1.91% and 2.43%, respectively, with inference times under one second. In contrast, the multilayer perceptron (MLP) struggled and failed to produce meaningful predictions. This suggests that simpler tree-based models can effectively and accurately predict physical simulation data. The comparable accuracy to physics-informed neural networks (PINNs) reported in the literature, combined with the fast prediction time, indicates that tree-based methods like XGBoost are attractive options for real-time engineering applications where computational time and resources are limited.

Compared to similar studies employing PINNs, such as [5] and [6], the tree-based models achieved similar error rates despite using simpler architectures. Unlike PINNs, which incorporate physics constraints during training, tree-based models rely purely on data-driven learning, likely contributing to their computational efficiency. Additionally, the simulation data itself implicitly encodes the underlying physics, potentially reducing or eliminating the need for explicit physics-informed models when the mathematical model is robust.

An important observation is that the prediction error tends to increase exponentially with the maximum temperature reached in the simulations. The rate of increase, r , was found to be 1.59 for XGBoost, 1.89 for Random Forest, and 1.26 for the MLP. This suggests that the models, particularly the tree-based ones, may struggle to accurately

capture the system’s behavior under extreme thermal conditions. Such regimes likely involve nonlinearities or physical effects underrepresented in the training data, reducing model reliability when extrapolating beyond the observed parameter space. In contrast, physics-informed models like PINNs, which directly incorporate governing equations, may maintain better accuracy under these extreme conditions by leveraging known physical constraints. Recognizing this limitation suggests that future work should explore new machine learning approaches, such as physics-informed tree-based models, that combine data-driven learning with physics awareness to improve both accuracy and generalization.

8. Conclusion

This study aimed to identify an optimal machine learning model for predicting physical data with reduced error and improved computational efficiency. The key findings show that tree-based models (XGBoost and Random Forest) successfully approximated the partial differential equation, achieving mean absolute percentage errors (MAPE) of 1.91% and 2.43%, respectively, while the neural network model (MLP) did not provide reliable predictions. When compared to physics-informed neural networks (PINNs), these tree-based models demonstrated similar error levels, suggesting they may offer a practical alternative for real-time engineering applications before resorting to more complex and computationally intensive methods such as PINNs.

The high accuracy of tree-based models is largely attributable to the robustness of the mathematical/physical model generating the data, which implicitly encodes the physics. However, an important limitation is that prediction error increases exponentially with maximum temperature, such that Absolute Error $\propto T^r$, with the rate r being 1.59 for XGBoost, 1.89 for Random Forest, and 1.26 for MLP. This indicates challenges in capturing extreme thermal behavior, although the MLP error appears to increase more slowly. In contrast, PINNs may better extrapolate and generalize for less well-defined models by explicitly incorporating physics knowledge.

Future work should focus on developing physics-informed tree-based models to further improve computational efficiency and generalization.

References

- [1] Bishop, C.M., 1995. Neural Networks for Pattern Recognition. Oxford University Press.
- [2] Breiman, L., 2001. Random forests. Machine Learning 45, 5–32. doi:10.1023/A:1010933404324.
- [3] Chen, T., Guestrin, C., 2016. Xgboost: A scalable tree boosting system. arXiv preprint arXiv:1603.02754 .
- [4] Evans, L.C., 2010. Partial Differential Equations. 2nd ed., American Mathematical Society. Section 2.3, p. 49.

- [5] Li, S., Wang, G., Di, Y., Wang, L., Wang, H., Zhou, Q., 2023. A physics-informed neural network framework to predict 3d temperature field without labeled data in process of laser metal deposition. *Engineering Applications of Artificial Intelligence* 120, 105908. URL: <https://doi.org/10.1016/j.engappai.2023.105908>, doi:10.1016/j.engappai.2023.105908.
- [6] Peng, S., Yang, S., Gao, B., Liu, W., Wang, F., Tang, Z., 2025. Prediction of 3d temperature field through single 2d temperature data based on transfer learning-based pinn model in laser-based directed energy deposition. *Journal of Manufacturing Processes* 88, 1017–1030. doi:10.1016/j.jmapro.2025.02.015.
- [7] Uhrich, B., Pfeifer, N., Schäfer, M., et al., 2024. Physics-informed deep learning to quantify anomalies for real-time fault mitigation in 3d printing. *Applied Intelligence* 54, 4736–4755. doi:10.1007/s10489-024-05402-4.